

Chapter 1

Geospatial Data Visualization with Python

Qiusheng Wu?? and Xinming Zhang??

^aDepartment of Geography and Sustainability, University of Tennessee, Knoxville

ABSTRACT

This chapter explores modern approaches to geospatial data visualization using Python libraries, with emphasis on fundamental visualization principles applied to Earth observation (EO) imagery analysis. We present a standardized framework for working with both vector and raster geospatial data through libraries like GeoPandas, Rasterio, Leafmap, and Geemap. The chapter progresses from fundamental visualization concepts to practical implementation, providing researchers, data scientists, and GIS professionals with the tools needed to effectively visualize and analyze geospatial information. Through practical examples and best practices rooted in established visualization principles, readers will learn to create dynamic, interactive, and information-rich visualizations that support better understanding of spatial relationships and patterns in Earth observation data.

KEYWORDS

Geospatial Visualization, Python, GeoPandas, Rasterio, Leafmap, Geemap, Interactive Mapping

1.1 INTRODUCTION

1.1.1 What is Visualization and Why Does it Matter?

Visualization is the process of transforming abstract data into visual representations that reveal patterns, relationships, and insights that would otherwise remain hidden in tables of numbers or complex datasets. At its core, effective visualization serves as a bridge between raw information and human understanding, leveraging our innate ability to process visual information rapidly and intuitively. As Edward Tufte, the pioneer of modern data visualization theory, eloquently stated: “*Excellence in statistical graphics consists of complex ideas communicated with clarity, precision, and efficiency.*” (?)

In the context of Earth observation imagery analysis, visualization plays multiple critical roles that extend far beyond simple data display. First, it serves as an **exploratory tool**, enabling researchers and analysts to identify spatial patterns, temporal trends, and anomalies across vast datasets that would be impossible to detect through numerical analysis alone. Second, visualization functions as a

communication medium, transforming complex geospatial analyses into accessible formats that can inform decision-makers, educate the public, and facilitate scientific collaboration. Third, it acts as a **validation mechanism**, allowing analysts to visually verify the results of automated algorithms and identify potential errors or artifacts in processed data.

1.1.2 The Unique Components of Earth Observation Visualization

Earth observation (EO) data visualization differs fundamentally from conventional data visualization in several key ways that shape both the techniques we use and the principles we apply:

1.1.2.1 *Spatial Context and Geographic Reference*

Unlike traditional statistical visualizations, EO data is inherently tied to specific locations on Earth's surface. This geographic anchoring requires visualization techniques that preserve spatial relationships and provide meaningful geographic context. The challenge lies in representing three-dimensional reality (latitude, longitude, and often elevation or time) on two-dimensional displays while maintaining spatial accuracy and interpretability.

1.1.2.2 *Map Projections and Coordinate Systems*

One of the most fundamental challenges in Earth observation visualization is the mathematical transformation required to represent Earth's curved surface on flat displays, a process handled through map projections. Every map projection involves compromises between different geometric properties: area, shape, distance, and direction. Understanding these trade-offs is crucial for effective EO visualization.

1.1.2.3 *Multi-dimensional and Multi-temporal Complexity*

Earth observation data often contains multiple spectral bands, temporal sequences, and derived indices, creating visualization challenges that extend beyond simple x-y plotting. A single satellite image might contain dozens of spectral bands, while time series analyses might involve thousands of images spanning decades. Effective visualization must help users navigate this complexity without overwhelming them with information.

1.1.2.4 *Scale Dependencies and Resolution Considerations*

EO data visualization must accommodate viewers operating at vastly different spatial and temporal scales—from global climate patterns spanning decades to local urban changes occurring over months. This requires adaptive visualization strategies that reveal relevant patterns at each scale while maintaining computational efficiency.

1.1.2.5 *Uncertainty and Data Quality Representation*

Earth observation data contains inherent uncertainties from atmospheric conditions, sensor limitations, and processing algorithms. Effective visualization must communicate not only what the data shows but also the confidence we should place in those observations.

1.1.3 Historical Evolution and Theoretical Foundations

The evolution of Earth observation visualization closely parallels the development of remote sensing technology, GIS software development, and general visualization theory. Edward Tufte's foundational principles (e.g., maximizing the data-ink ratio, avoiding chartjunk, and ensuring graphical integrity) (?) translate directly to geospatial visualization with important adaptations for spatial data.

Tufte's principle of **graphical excellence** in EO visualization means creating displays that efficiently reveal spatial patterns while avoiding unnecessary visual elements that distract from the underlying geographic phenomena. His emphasis on **data density** encourages the development of techniques that can display rich, multi-dimensional information without creating visual clutter—a particular challenge when working with hyperspectral imagery or dense time series.

1.1.4 Python for Geospatial Data Visualization

The ability to effectively visualize geospatial data has become increasingly important as organizations collect and analyze larger volumes of location-based information. Whether tracking environmental changes, optimizing supply chains, analyzing demographic patterns, or responding to natural disasters, the capacity to represent spatial data visually provides critical insights that might otherwise remain hidden in tables of coordinates and attributes.

Python has emerged as a leading programming language for geospatial data science due to its readability, extensive ecosystem of libraries, and strong community support. The rise of artificial intelligence and deep learning has further cemented Python's dominance in geospatial applications. The language's widespread adoption in the AI community, powered by frameworks like TensorFlow, PyTorch, and scikit-learn, has created natural synergies with geospatial analysis. These machine learning capabilities enable automated feature extraction from satellite imagery (??), predictive modeling of environmental phenomena (?), and sophisticated pattern recognition (?) in spatial data—tasks that were previously labor-intensive or computationally prohibitive.

Four Python libraries in particular have transformed how practitioners work with geospatial data:

1. **GeoPandas** extends the popular pandas data analysis library to support geometric data types and spatial operations, making it ideal for working with vector data like points, lines, and polygons (?).

2. **Rasterio** provides a high-level interface to GDAL (Geospatial Data Abstraction Library) for reading and writing raster datasets such as satellite imagery, digital elevation models, and other gridded data (?).
3. **Leafmap** builds on established mapping libraries to offer a user-friendly interface for creating interactive web maps and visualizing both vector and raster data (?).
4. **Geemap** enables easy access to and analysis of Google Earth Engine’s vast repository of geospatial data, allowing Python users to harness the power of cloud computing for large-scale geospatial analysis and visualization (?).

Together, these tools enable practitioners to process, analyze, and visualize geospatial data with unprecedented ease and flexibility. They support workflows ranging from exploratory data analysis to the creation of publication-quality static maps and interactive web applications.

This chapter explores how these libraries can be used individually and in combination to create effective geospatial visualizations. We begin with the fundamental concepts and data structures that underpin geospatial data in Python, then progress to specific visualization techniques for both vector and raster data. Throughout, we emphasize practical implementation with clear code examples and best practices to guide readers in developing their own visualization workflows.

1.2 METHODOLOGY

This section introduces the fundamental principles and practical application of geospatial data visualization using Python. We’ll guide you through the essential concepts, core Python libraries, and the underlying approaches to effectively represent spatial data. Designed for readers without a background in Geoscience or Remote Sensing, this guide emphasizes a clear, step-by-step explanation alongside practical code examples. Our primary goal is to equip you with the knowledge and skills necessary to create insightful and informative geospatial visualizations with a few lines of code.

1.2.1 Fundamental Principles for Geospatial Data Visualization

Before delving into practical implementation, it is essential to understand the core principles that govern effective geospatial data visualization. These principles build upon Edward Tufte’s foundational visualization concepts while addressing the unique challenges of spatial data representation.

Spatial Context Preservation: Unlike conventional data visualization, geospatial visualizations must maintain geographic relationships and provide meaningful spatial context. This requires careful consideration of map projections, coordinate systems, and the trade-offs between different geometric properties (area, shape, distance, direction) when representing Earth’s curved surface on flat displays.

Multi-dimensional Information Management: Earth observation data often contains multiple spectral bands, temporal sequences, and derived indices. Effective visualization must help users navigate this complexity through techniques such as band combinations, temporal animations, and interactive layer controls, without overwhelming the viewer with information density.

Scale-Adaptive Design: Geospatial visualizations must accommodate viewers operating at vastly different spatial and temporal scales. This requires adaptive strategies that reveal relevant patterns at each scale while maintaining computational efficiency and visual clarity.

Uncertainty Communication: Earth observation data contains inherent uncertainties from atmospheric conditions, sensor limitations, and processing algorithms. Effective visualization must communicate not only what the data shows but also the confidence we should place in those observations, often through techniques such as confidence intervals, error bars, or transparency mapping.

Graphical Excellence for Spatial Data: Applying Tufte’s principles to geospatial contexts means maximizing the spatial-data-ink ratio by eliminating unnecessary visual elements (chartjunk) that distract from geographic phenomena, while ensuring that color schemes, symbols, and styling choices enhance rather than obscure spatial patterns and relationships.

These principles guide our approach throughout this methodology section, informing decisions about data formats, visualization techniques, and interactive design choices that support effective communication of geospatial insights.

1.2.2 Setting Up the Python Environment

Before embarking on geospatial visualization, it’s crucial to create a virtual environment containing the necessary libraries. This section outlines two recommended approaches for creating such an environment: using the conda package manager and the newer, high-performance uv package manager.

1.2.2.1 Using conda for Environment Setup

Conda is a popular package manager that handles library dependencies efficiently, especially for complex scientific packages with compiled components. The following steps will guide you through setting up a dedicated geospatial environment using conda:

1. **Install Miniconda or Anaconda:** Begin by installing either Miniconda (a minimal version) or Anaconda (a comprehensive distribution with many pre-installed packages). Miniconda is recommended for users who prefer a lightweight installation, while Anaconda is suitable for those who want a full-featured environment out of the box.
2. **Create a new conda environment:** Open your terminal (or Anaconda Prompt on Windows) and execute the following command to create a new

environment specifically for your geospatial projects:

```
1 conda create -n geoenv python
```

This command creates a new conda environment named `geoenv` with the latest version of Python. You can specify a particular Python version if needed (e.g., `python=3.12`).

- 3. Activate the environment:** Once the environment is created, you need to activate it to start using it. Run the following command:

```
1 conda activate geoenv
```

This command activates the `geoenv` environment, allowing you to install and manage packages within this isolated space.

- 4. Install geospatial libraries:** Leverage the conda-forge channel, a community-driven repository known for providing up-to-date and well-maintained geospatial packages. Use the following command to install essential libraries:

```
1 conda install -c conda-forge geopandas rasterio leafmap geemap
  maplibre fiona localtileserver mapclassify kaleido
```

This command installs `geopandas` (for vector data), `rasterio` (for raster data), `leafmap` (for interactive maps), `geemap` (for Google Earth Engine integration), `maplibre` (for 3D visualization), `fiona` (for file I/O), `localtileserver` (for local tile servers), `mapclassify` (for classification), and `kaleido` (for creating static images from interactive plots). The `-c conda-forge` flag specifies that packages should be installed from the conda-forge channel, which typically has the most up-to-date versions of geospatial packages with proper dependency management.

1.2.2.2 Using uv for Environment Setup

`uv` is a modern, extremely fast Python package installer and resolver. Its efficiency makes it an attractive alternative to conda, particularly for projects where speed is a concern. The following steps detail how to set up a geospatial environment using `uv`:

- 1. Install uv:** If you haven't already, install `uv` using the provided installation scripts for your operating system:

```
1 # For Windows
2 powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv
  /install.ps1 | iex"
3
4 # For macOS and Linux
5 curl -LsSf https://astral.sh/uv/install.sh | sh
```

- 2. Create a virtual environment:** `uv` creates standard Python virtual environments. Create one in your project directory:

```
1 uv venv
```

This command creates a new virtual environment with the default Python version in the current directory. The virtual environment will be stored in a hidden folder named `.venv` by default. You may need to show hidden files to see it. You can also specify a particular Python version if needed (e.g., `uv venv --python 3.12`).

3. **Install geospatial libraries:** Use `uv pip` to install the required geospatial libraries within the activated virtual environment:

```
1 uv pip install geopandas rasterio leafmap geemap maplibre fiona
   localtileserver mapclassify kaleido
```

Unlike `conda`, which manages complex binary dependencies, `uv` primarily manages Python packages. However, its significantly faster installation speeds (often 10x faster than traditional `pip`) make it an excellent choice for projects where the underlying system dependencies are already available.

1.2.2.3 Package Installation Notes

Installing geospatial packages can sometimes present challenges due to their reliance on system-level dependencies. Consider these points based on your operating system:

- **Windows:** `conda` generally offers a more streamlined experience for Windows users, as it often provides pre-compiled binaries that handle system-level dependencies automatically.
- **Linux:** You may need to install additional system libraries manually. On Debian-based systems like Ubuntu, the following command can often resolve dependency issues for geospatial libraries:

```
1 sudo apt-get install libgdal-dev libproj-dev proj-bin libgeos-
   dev
```

- **macOS:** Using `conda` is frequently the easiest approach on macOS. Alternatively, Homebrew can be used to install dependencies, but `conda` often provides a more self-contained solution.

1.2.2.4 JupyterLab Setup

For an improved interactive development experience, we recommend using JupyterLab over the classic Jupyter Notebook. JupyterLab provides a more modern and feature-rich environment for working with geospatial data and visualizations. Open a terminal and run the following command to install JupyterLab:

```
1 conda install -c conda-forge jupyterlab # or: uv pip install
   jupyterlab
```

Then, launch JupyterLab using the following command:

```
1 jupyter lab
```

Once JupyterLab is running, you can access it through your web browser. Create a new notebook to start working with geospatial data. JupyterLab's interface allows you to easily manage multiple notebooks, text editors, and terminal sessions within a single window, enhancing your productivity and workflow.

1.2.2.5 Source Code Availability

The source code for this chapter is available on GitHub at the following link:

- <https://github.com/opengeos/geospatial-dataviz-python>

You can clone the repository and run the Jupyter notebooks locally or on Google Colab to follow along with the examples and exercises presented in this chapter.

If you are using Google Colab, you can directly install the required libraries by running the following command in a code cell:

```
1 %pip install geopandas rasterio leafmap geemap maplibre fiona
   localtileserver mapclassify kaleido
```

1.2.2.6 Testing Your Installation

To ensure that your Python environment is correctly set up with all the geospatial libraries, let's create a new code block in a Jupyter Notebook and run a simple test script. This script will verify that the required libraries can be imported and that basic functionality is working.

```
1 # Test imports
2 import geopandas as gpd
3 import rasterio
4 import leafmap
5 import geemap
6
7 # Print versions
8 print(f"GeoPandas version: {gpd.__version__}")
9 print(f"Rasterio version: {rasterio.__version__}")
10 print(f"Leafmap version: {leafmap.__version__}")
11 print(f"Geemap version: {geemap.__version__}")
12
13 # Create a simple map
14 m = leafmap.Map()
15 m
```

If the script executes without any errors and displays an interactive map, it confirms that your environment is properly configured and ready for geospatial data visualization. The import statements load the necessary libraries, the print statements display the version numbers, and the `leafmap.Map()` call creates a basic interactive map, which is the ultimate test of your setup. The interactive map should appear directly in your notebook, with zoom controls and a basemap showing a world map.

1.2.3 Fundamental Geospatial Data Types

Geospatial data, the information tied to specific locations on Earth, comes in two primary flavors: vector and raster. Understanding these data types is crucial for choosing the right tools and techniques for visualization and analysis.

1.2.3.1 Vector Data

Vector data represents geographic features as discrete geometric objects. Think of it like drawing shapes on a map. There are three main types of vector data:

- **Points:** Represent single locations with a specific latitude and longitude. Examples include cities, landmarks, or the location of individual trees.
- **Lines:** Represent linear features like roads, rivers, or power lines. They are defined by a series of connected points.
- **Polygons:** Represent areas with boundaries, such as countries, lakes, or buildings. They are defined by a closed loop of connected points.

Vector data is typically stored in file formats such as:

- **Shapefiles:** A common, older format. It consists of multiple files with different extensions (e.g., .shp, .shx, .dbf) that together represent the geometric and attribute data.
- **GeoJSON:** A lightweight, text-based format. It's easy to read and write, making it popular for web applications. GeoJSON represents geographic features as JSON objects, which can be easily manipulated in web-based applications.
- **GeoPackage:** A modern, single-file format that supports both vector and raster data. It is an open standard format that can store multiple layers of geospatial data in a single file, making it efficient for storage and sharing.
- **GeoParquet:** A column-oriented format for efficient storage and querying. It is designed for high-performance geospatial data storage and retrieval, especially for large datasets. GeoParquet is built on top of the Apache Parquet format, which is optimized for efficient data compression and encoding.

These formats contain both the geometric information (the points, lines, or polygons) and associated attributes (e.g., the name of a city, the length of a road, the population of a country).

1.2.3.2 Raster Data

Raster data represents continuous surfaces as a grid of cells or pixels. Imagine a photograph of the Earth taken from space. Each pixel in the image has a value representing a specific characteristic of that location. Common examples include:

- **Satellite imagery:** Images captured by satellites, showing the Earth's surface in different wavelengths of light.

- **Elevation models:** Digital representations of terrain, where each pixel's value represents the elevation at that location.
- **Climate data:** Data representing temperature, rainfall, or other climate variables, where each pixel's value represents the value of that variable at that location.

Raster data is typically stored in file formats such as:

- **GeoTIFF:** A widely used format for storing georeferenced raster data.
- **NetCDF:** A format commonly used for storing scientific data, including climate and oceanographic data.
- **Cloud-Optimized GeoTIFF (COG):** A variant of GeoTIFF that is optimized for cloud storage and access, enabling efficient retrieval of specific parts of the image.

1.2.4 Working with Vector Data: GeoPandas

GeoPandas is a powerful Python library that extends the capabilities of the popular pandas library to handle geospatial vector data. Essentially, it adds a “geometry” column to pandas DataFrames, turning them into GeoDataFrames that support spatial operations.

1.2.4.1 Loading Vector Data

The first step in working with vector data is loading it into a GeoDataFrame. GeoPandas can read from various file formats including Shapefile, GeoJSON, and GeoPackage. Let's start by importing the library:

```
1 import geopandas as gpd
```

Next, we'll load a GeoJSON file containing data about New York City boroughs from a remote URL:

```
1 vector_path = "https://github.com/opegeos/datasets/releases/
  download/vector/nybb.geojson"
2 gdf = gpd.read_file(vector_path)
```

The `read_file()` function automatically detects the file format based on the file extension or URL path. It supports various geospatial formats beyond GeoJSON, including Shapefile, GeoPackage, and more.

1.2.4.2 Exploring and Manipulating Vector Data

Once your data is loaded into a GeoDataFrame, you can explore its contents and structure:

```
1 gdf.head()
```

This displays the first five rows of the GeoDataFrame, showing both the attribute columns (BoroCode, BoroName) and the special geometry column

that contains the spatial data. The output would show a table with borough information and a geometry column containing complex polygon objects.

To understand what coordinate system the data uses, check the Coordinate Reference System (CRS):

```
1 print(gdf.crs)
```

The CRS defines how the coordinates in the geometry column relate to locations on the Earth's surface. It's crucial to know the CRS when combining multiple datasets or performing spatial operations.

GeoPandas provides many spatial operations. Let's create a buffer around each borough:

```
1 buffered = gdf.buffer(distance=1000)
```

This operation creates a new geometry collection where each borough polygon has been expanded outward by 1,000 meters (1 kilometer) in all directions. Buffering is a common operation in spatial analysis. The `buffer()` method creates expanded polygons by drawing a circle of the specified radius around each vertex of the original geometry and then creating a smooth outline that encompasses all these circles. The result is a new set of geometries that are larger than the originals but maintain a similar shape.

1.2.4.3 Visualizing Vector Data

GeoPandas makes it easy to create static maps from your vector data using the `plot()` method:

```
1 gdf.plot(figsize=(10, 6))
```

This creates a simple map showing the borough boundaries. The `figsize` parameter sets the size of the resulting figure in inches (width, height). The output is a basic map with each borough shown as a filled polygon in the same color.

For more informative visualizations, we can create thematic maps where features are colored according to their attributes:

```
1 ax = gdf.plot(column='BoroName', cmap='viridis', legend=True,
2               figsize=(10, 6), edgecolor='black')
3 ax.get_legend().set_loc('upper left')
4 plt.savefig("nyc.svg", format="svg", bbox_inches='tight')
5 plt.show()
```

This code creates a choropleth map (Figure 12.1) where:

- Each borough is colored according to its name (`column='BoroName'`)
- The color scheme uses the viridis colormap (`cmap='viridis'`)
- Borough boundaries are outlined in black (`edgecolor='black'`)
- A legend is added showing which color corresponds to which borough (`legend=True`)

- The legend is positioned in the upper left corner of the map

The `GeoDataFrame`'s `plot()` method returns a matplotlib axes object (`ax`), which we can further customize. Here, we reposition the legend from the default upper-right position to the upper-left position to avoid covering important parts of the map.

[Insert Figure 12.1 here]

1.2.5 Working with Raster Data: Rasterio

Rasterio is a Python library that provides a clean and efficient way to read, manipulate, and write raster data. It's built on top of GDAL, a powerful geospatial data processing library, but offers a more Pythonic interface.

1.2.5.1 Loading Raster Data

To work with raster data, first import the necessary libraries:

```
1 import rasterio
2 import rasterio.plot
3 import matplotlib.pyplot as plt
```

Now let's open a Digital Elevation Model (DEM) raster file:

```
1 raster_path = (
2     "https://github.com/opengeos/datasets/releases/download/raster
3     /dem_90m.tif"
4 )
5 src = rasterio.open(raster_path)
```

This above code:

1. Defines the URL path to a GeoTIFF file containing elevation data
2. Uses `rasterio.open()` to load the raster dataset
3. Assigns the opened dataset to the variable `src`

Unlike `GeoPandas`, which loads the entire dataset into memory, `Rasterio` keeps a connection to the data source and only reads data as needed, making it more memory-efficient for large raster datasets.

1.2.5.2 Accessing Raster Metadata

Raster files contain important metadata about their content and structure:

```
1 src.meta
```

This displays a dictionary containing essential information about the raster, including:

- width and height: The dimensions of the raster in pixels
- count: The number of bands (channels) in the raster
- dtype: The data type of the pixel values (e.g., `float32`, `uint8`)

- `crs`: The coordinate reference system
- `transform`: The affine transformation that maps pixel coordinates to geographic coordinates

To get the spatial resolution (pixel size), use:

```
1 src.res
```

This returns a tuple of (`x_resolution`, `y_resolution`), typically in the units of the CRS (often meters). For example, (90.0, 90.0) would indicate 90-meter pixels in both directions.

1.2.5.3 Visualizing Raster Data

Rasterio provides functions for visualizing raster data using Matplotlib:

```
1 fig, ax = plt.subplots(figsize=(8, 8))
2 rasterio.plot.show(src, cmap="terrain", ax=ax, title="Digital
  Elevation Model (DEM)")
3 plt.show()
```

This above code:

1. Creates a matplotlib figure and axes with a size of 8×8 inches
2. Uses `rasterio.plot.show()` to display the raster data on the axes
3. Sets the color palette to `terrain`, which is suitable for elevation data (using shades of green for low elevations and brown/white for higher elevations)
4. Adds a title to the plot
5. Displays the plot (Figure 12.2)

[Insert Figure 12.2 here]

The `rasterio.plot.show()` function automatically handles the transformation between the raster's internal coordinate system and the plot's coordinate system, ensuring the data is displayed correctly.

1.2.5.4 Processing Raster Data

Raster data can be processed and manipulated as NumPy arrays, enabling powerful calculations and transformations:

```
1 dem_band = src.read(1)
2 mountain = dem_band > 3000
```

The first line reads the first band (indexed from 1) of the raster into a NumPy array called `dem_band`. For a single-band raster like a DEM, there's only one band containing elevation values.

The second line creates a boolean mask called `mountain` where:

- Each element is `True` where the corresponding elevation value is greater than 3000 meters
- Each element is `False` where the elevation is 3000 meters or less

This simple thresholding operation creates a binary mask identifying high-elevation areas (mountains). Similar techniques can be used to identify other features in raster data, such as water bodies, forested areas, or urban development.

1.2.5.5 Visualizing Processed Raster Data

We can visualize the results of our processing:

```
1 fig, ax = plt.subplots(figsize=(8, 8))
2 rasterio.plot.show(mountain, ax=ax, title="Mountains (Elevation >
3   3,000 m)", cmap="gray")
4 plt.show()
```

This displays our binary mountain mask as a black and white image, where:

- White pixels represent mountainous areas (elevation > 3000m)
- Black pixels represent non-mountainous areas

This simple visualization effectively highlights regions of high elevation, making it easy to identify mountain ranges. The `cmap="gray"` parameter specifies a grayscale colormap, appropriate for binary data. You can also use any other colormap supported by Matplotlib, such as "viridis" or "plasma", to create more visually appealing representations.

1.2.6 Interactive Mapping: Leafmap

GeoPandas and Rasterio are excellent for static maps and data manipulation, but for interactive web maps, we turn to Leafmap. Leafmap is a Python library designed for creating interactive maps with minimal code. It allows you to visualize both vector and raster data in a Jupyter environment. Leafmap is built on top of popular libraries like `ipyleaflet` and `folium`. It provides a high-level unified API for creating maps, adding layers, and interacting with map elements.

1.2.6.1 Creating a Base Map

Let's start by importing the library and creating a simple interactive map:

```
1 import leafmap
```

Now we can create a basic interactive map:

```
1 m = leafmap.Map(center=[20, 0], zoom=2)
2 m
```

This above code:

1. Creates a new Leafmap map object
2. Centers it at latitude 20°, longitude 0° (near the equator in Africa)
3. Sets the initial zoom level to 2 (showing most of the world)
4. Displays the map in the notebook

The resulting map is fully interactive - you can zoom in/out, pan around, and view the coordinates of any location. By default, Leafmap uses the OpenStreetMap basemap, which provides a simple view of the world with basic features like country boundaries, major cities, and transportation networks.

1.2.6.2 Adding Basemaps

Leafmap provides access to numerous basemaps from providers like Esri:

```
1 m.add_basemap("Esri.WorldImagery")
```

This adds the Esri World Imagery basemap, which provides high-resolution satellite imagery for the entire world. The map now displays detailed aerial/satellite imagery instead of the default OpenStreetMap style. Users can select different basemaps using the toolbar button that appears in the top-right corner of the map. Note that new layers will be added to map displayed previously, so you may need to scroll up to see newly added basemap.

1.2.6.3 Adding Vector Data

You can add vector data to your Leafmap map using the `add_vector()` function:

```
1 vector_path = "https://github.com/opengeos/datasets/releases/download/us/us_state_20m.geojson"
2 m.add_vector(vector_path, layer_name="US States")
```

The `add_vector()` function performs several operations behind the scenes:

- Downloads the GeoJSON data from the specified URL
- Converts it to a format that can be displayed on web maps
- Adds it as an interactive layer to the map
- Automatically styles the features (states in this case) with a default fill color and outline

The resulting map shows US state boundaries overlaid on the basemap. You can hover your mouse on any state to view its attributes (properties from the GeoJSON file) in a popup in the lower-right corner. You can also toggle the visibility of this layer using the layer control panel.

1.2.6.4 Adding Raster Data

Leafmap can also display raster data as an overlay:

```
1 raster = "https://github.com/opengeos/datasets/releases/download/raster/LC09_039035_20240708_90m.tif"
2 m.add_raster(raster, indexes=[5, 4, 3], layer_name="Landsat 9")
```

The `add_raster()` function:

1. Retrieves the raster data from the specified URL
2. Applies the band combination specified by indexes

3. Constructs a tile layer from the raster data
4. Adds it as an interactive layer to the map

This particular band combination (5-4-3) makes vegetation appear bright red, urban areas appear cyan, and water appears dark blue or black. Different band combinations highlight different features on the Earth's surface.

1.2.7 Cloud-based Geospatial Analysis: Geemap

Geemap bridges the gap between Python and the Google Earth Engine (GEE) platform (?), enabling users to access and process massive geospatial datasets in the cloud. GEE hosts petabytes of satellite imagery and other geospatial data, combined with powerful computational resources for processing this data at scale.

1.2.7.1 Setting up Geemap

Before using Geemap, you need to initialize it and authenticate with Google Earth Engine:

```
1 import ee
2 import geemap
```

Initialize the Earth Engine API:

```
1 geemap.ee_initialize()
```

This above line:

1. Checks if you're already authenticated with Google Earth Engine
2. If not, initiates the authentication process
3. Establishes a connection to the Earth Engine servers

The first time you run this, you'll be prompted to authenticate with your Google account that has Earth Engine access. After authentication, you can create a map:

```
1 m = geemap.Map(center=[20, 0], zoom=2)
2 m
```

This creates an interactive map similar to Leafmap, but with added capabilities for working with Earth Engine data. The map includes Earth Engine's layer control system in addition to the standard mapping interface.

1.2.7.2 Accessing and Visualizing Satellite Imagery

Geemap allows you to access and visualize satellite imagery from Google Earth Engine's vast data catalog:

```
1 # Load Landsat 8 collection
2 landsat = ee.ImageCollection("LANDSAT/LC08/C02/T1_L2") \
```



```

3 .filterDate('2025-01-01', '2025-12-31') \
4 .filterBounds(ee.Geometry.Point(-122.085, 37.422)) \
5 .sort('CLOUD_COVER') \
6 .first()
7
8 # Define visualization parameters
9 vis_params = {
10     'bands': ['SR_B4', 'SR_B3', 'SR_B2'],
11     'min': 7000,
12     'max': 15000
13 }
14
15 # Add the image to the map
16 m.add_layer(landsat, vis_params, "Landsat 8")
17 m.center_object(landsat, 8)

```

This code performs a simple but powerful Earth Engine workflow:

1. Data Selection:

- Accesses the Landsat 8 Collection 2 Level-2 dataset (LANDSAT/LC08/C02/T1_L2)
- Filters for images from 2025 (note: you should adjust this to use dates that have actual data)
- Filters for images that include the point near Google's headquarters in Mountain View, CA
- Sorts by cloud cover percentage (ascending, so least cloudy first)
- Takes the first (least cloudy) image from the filtered collection

2. Visualization Setup:

- Defines how the image should be displayed:
- 'bands': Uses bands 4 (red), 3 (green), and 2 (blue) for a natural color display
- 'min' and 'max': Sets the range for scaling pixel values to display colors
- These values are specific to the surface reflectance data in Landsat 8

3. Map Display:

- Adds the image to the map with the specified visualization parameters
- Centers the map on the image at zoom level 8

The key advantage here is that all data remains in Google's cloud. Your browser only receives the small tiles needed for display, not the full multi-gigabyte satellite image.

To inspect the metadata of the Landsat image:

```
1 landsat
```

This displays a representation of the Earth Engine Image object, showing properties like acquisition date, cloud cover percentage, satellite path/row, and other metadata.

1.2.7.3 Computing and Visualizing Indices

Geemap allows you to perform calculations on satellite imagery directly in the cloud:

```

1 ndvi = landsat.normalizedDifference(['SR_B5', 'SR_B4']).rename('
   NDVI')
2 ndvi_vis = {
3     'min': -1,
4     'max': 1,
5     'palette': ['blue', 'white', 'green']
6 }
7 m.add_layer(ndvi, ndvi_vis, 'NDVI')
8 m

```

This code:

1. Calculates NDVI:

- Uses the `normalizedDifference()` method to compute $(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$
- NIR is Band 5 in Landsat 8
- Red is Band 4 in Landsat 8
- This calculation is performed for every pixel in the image on Google's servers
- The result is renamed to NDVI for clarity

2. Defines Visualization:

- NDVI ranges from -1 to 1, so those are set as the min/max values
- A color palette is defined where:
 - Blue represents negative values (typically water)
 - White represents values near zero (typically built-up areas, bare soil)
 - Green represents positive values (typically healthy vegetation)

3. Adds to the Map:

- Adds the NDVI layer to the map with the specified visualization parameters
- Names the layer 'NDVI' in the layer control panel

This computation would require downloading gigabytes of satellite data if performed locally, but with Earth Engine, it runs in seconds on Google's infrastructure. The NDVI visualization makes vegetation health immediately apparent - brighter green areas have higher photosynthetic activity indicating healthier or denser vegetation.

1.3 STATE OF THE ART AND CURRENT DEVELOPMENTS

Geospatial data visualization with Python has undergone a remarkable transformation, evolving from rudimentary mapping functionalities to a sophisticated ecosystem of interactive and cloud-native tools. In its early stages (2008-2013), libraries like Matplotlib provided foundational mapping capabilities, albeit with

limitations in interactivity and ease of use, often requiring significant expertise. A pivotal moment arrived in the middle period (2014-2019) with the emergence of GeoPandas, initially released in 2013, which seamlessly integrated spatial data handling with the powerful data manipulation capabilities of the pandas library.

Cartopy, introduced in 2014, further enhanced Python's mapping capabilities by providing advanced geospatial plotting features. The introduction of Folium in 2013 and ipyleaflet in 2016 marked a significant leap forward, enabling the creation of interactive web maps using Leaflet.js, a popular JavaScript library for mobile-friendly interactive maps. These libraries allowed users to create visually appealing maps with minimal code, making geospatial visualization more accessible to a broader audience.

The current era (2020-present) is marked by a surge of user-friendly libraries such as leafmap, geemap, pydeck, keplerGL, maplibre, and lonboard, alongside improved integration between these tools. Furthermore, the adoption of cloud-native geospatial formats like Cloud Optimized GeoTIFF (COG), GeoParquet, and SpatioTemporal Asset Catalogs (STAC) has revolutionized data storage, access, and dissemination.

A key trend driving this evolution is the increasing prevalence of interactive and web-based visualizations. Static maps are increasingly being augmented or replaced by dynamic, browser-based tools like folium, ipyleaflet, and pydeck, which generate interactive maps that can be readily embedded within web applications, Jupyter notebooks, and dashboards. Emerging platforms like Hugging Face, Streamlit, Solara, and PyCafe are further expanding options for hosting geospatial data and visualizations. Complementing this is the rise of cloud-native geospatial approaches. Cloud Optimized GeoTIFFs (COGs) enable efficient access to portions of large raster datasets, allowing visualization without the need for complete downloads. SpatioTemporal Asset Catalogs (STAC) standardize the indexing and discovery of geospatial data, simplifying the process of finding and visualizing relevant data from extensive collections. Moreover, visualization platforms are increasingly integrating with cloud services for on-demand processing, utilizing serverless computing and formats like PMTiles to efficiently handle large datasets before visualization, showcasing a move towards scalable and accessible geospatial insights.

1.4 APPLICATION EXAMPLES

In this section, we explore practical applications of geospatial analysis techniques using Python. We will demonstrate how to leverage the capabilities of various libraries to perform tasks such as land cover analysis, building height estimation, and 3D visualization of building data. Each example provides a step-by-step guide to implementing the techniques, along with explanations of the underlying concepts and code. These examples serve as a foundation for understanding how to apply geospatial analysis in real-world scenarios.

1.4.1 Required Packages

Let's start by installing the necessary packages for this section. Execute the code below to install the required packages in your Python environment:

```
1 %pip install maplibre rasterstats overturemaps rioxarray planetary
  -computer kaleido
```

This installation command adds specialized packages needed for our advanced examples:

- maplibre: For 3D map visualization
- rasterstats: For computing zonal statistics (calculations on raster data within vector boundaries)
- overturemaps: A client for accessing Overture Maps building footprint data
- rioxarray: For advanced raster analysis that combines rasterio with xarray
- planetary-computer: For accessing Microsoft's Planetary Computer data catalog
- kaleido: For creating static images from interactive plots

1.4.2 Land Cover Analysis with Earth Engine

In this section, we'll analyze land cover data from the National Land Cover Database (NLCD), which classifies land cover across the United States into different categories such as forest, urban, cropland, and water.

First, we import the required libraries:

```
1 import ee
2 import geemap
```

1.4.2.1 Visualizing Land Cover Data

Let's create an interactive map to visualize the NLCD land cover data across the United States:

```
1 # Create an interactive map
2 m = geemap.Map(center=[40, -100], zoom=4)
3
4 # Add NLCD data
5 dataset = ee.Image("USGS/NLCD_RELEASES/2019_REL/NLCD/2019")
6 landcover = dataset.select("landcover")
7 m.add_layer(landcover, {}, "NLCD 2019")
8
9 # Add US census states
10 states = ee.FeatureCollection("TIGER/2018/States")
11 style = {"fillColor": "00000000"}
12 m.add_layer(states.style(**style), {}, "US States")
13
14 # Add NLCD legend
15 m.add_legend(title="NLCD Land Cover", builtin_legend="NLCD")
16 m
```

The resulting interactive map displays (Figure 12.3) land cover patterns across the entire United States, with state boundaries overlaid for reference. Users can zoom and pan to explore different regions and use the legend to identify land cover types.

[Insert Figure 12.3 here]

1.4.2.2 Calculating Land Cover Statistics

Next, we calculate the area of each land cover type across the United States:

```
1 df = geemap.image_area_by_group(
2     landcover, scale=1000, denominator=1e6, decimal_places=4,
3     verbose=True
4 )
df
```

This function performs a comprehensive spatial analysis:

1. It calculates the area occupied by each unique value (land cover class) in the landcover image
2. The scale parameter (1000) determines the spatial resolution in meters at which to perform the calculation
3. The denominator (1e6) converts square meters to square kilometers
4. The result is a DataFrame showing each land cover class, its area in square kilometers, and its percentage of the total area

The resulting DataFrame shows quantitative information about land cover distribution, such as how much of the US is covered by forests, cropland, developed areas, etc.

Let's save these results to a CSV file for further analysis:

```
1 df.to_csv('nlcd_stats.csv')
```

This saves the statistical results to a CSV file, allowing for further analysis in spreadsheet software or other data analysis tools.

1.4.2.3 Adding Human-Readable Land Cover Class Names

The NLCD classification uses numeric codes that aren't immediately intuitive. Let's create a legend mapping these codes to descriptive names:

```
1 legend = {
2     '11': 'Open Water',
3     '12': 'Perennial Ice/Snow',
4     '21': 'Developed, Open Space',
5     '22': 'Developed, Low Intensity',
6     '23': 'Developed, Medium Intensity',
7     '24': 'Developed, High Intensity',
8     '31': 'Barren Land (Rock/Sand/Clay)',
9     '41': 'Deciduous Forest',
10    '42': 'Evergreen Forest',
11    '43': 'Mixed Forest',
```

```

12     '51': 'Dwarf Scrub',
13     '52': 'Shrub/Scrub',
14     '71': 'Grassland/Herbaceous',
15     '72': 'Sedge/Herbaceous',
16     '73': 'Lichens',
17     '74': 'Moss',
18     '81': 'Pasture/Hay',
19     '82': 'Cultivated Crops',
20     '90': 'Woody Wetlands',
21     '95': 'Emergent Herbaceous Wetlands'
22 }

```

This dictionary maps each NLCD class code to its official name. For example, code '11' represents 'Open Water' (lakes, rivers, oceans), while code '42' represents 'Evergreen Forest'.

Now we add these descriptive names to our DataFrame:

```

1 df['class'] = df.index.map(legend)
2 df

```

This creates a new column called `class` in our DataFrame, containing the descriptive name corresponding to each numeric class code. This makes the data more interpretable and ready for visualization.

1.4.2.4 Visualizing Land Cover Distribution

Data visualization helps us understand patterns and proportions more intuitively. Let's create a pie chart:

```

1 fig = geemap.pie_chart(df, names="class", values="area", height
2     =500)
3 fig.write_image("gee_nlcd_pie_chart.svg", width=900)
4 fig

```

This function creates an interactive pie chart where:

- Each slice represents a land cover class
- The size of each slice is proportional to its area
- The labels show the descriptive names from our legend
- The height parameter sets the chart's height in pixels

This visualization (Figure 12.4) immediately reveals which land cover types dominate the US landscape. It shows that shrub/scrub, croplands, and grasslands occupy the largest portions of the country.

[Insert Figure 12.4 here]

1.4.2.5 Analyzing Land Cover by State

While national statistics provide an overview, analyzing land cover by state reveals regional patterns and differences:

```

1 stats = geemap.zonal_stats_by_group(

```

```

2     landcover,
3     states,
4     stat_type="SUM",
5     denominator=1e6,
6     decimal_places=2,
7     return_fc=True,
8 )

```

This powerful function performs a complex geospatial analysis:

1. It overlays the NLCD landcover data with state boundaries
2. For each state, it calculates the area of each land cover class
3. The results are calculated in Earth Engine's cloud infrastructure
4. Setting `return_fc=True` returns an Earth Engine `FeatureCollection`

This analysis would be computationally intensive to perform locally, especially with high-resolution data like NLCD, but Earth Engine makes it efficient by distributing the computation across Google's infrastructure.

Let's convert the Earth Engine feature collection to a pandas `DataFrame` for easier analysis:

```

1 df_stats = geemap.ee_to_df(stats)
2 df_stats.head()

```

This converts the Earth Engine `FeatureCollection` to a pandas `DataFrame`, bringing the data into a tabular format that's easier to work with for further analysis.

```

1 fig = geemap.bar_chart(
2     df_stats,
3     x="STUSPS",
4     y="Class_82",
5     title="Cropland Area by State",
6     x_label="State",
7     y_label="Area (km2)",
8 )
9 fig.write_image("gee_nlcd_bar_chart.svg", width=900, height=350)
10 fig

```

This creates a bar chart showing:

- States (using their USPS abbreviations) on the x-axis
- Cropland area (Class 82 in NLCD) on the y-axis
- Appropriate title and axis labels

This visualization (Figure 12.5) reveals which states have the most cropland, providing insights into the geographic distribution of agricultural activity across the United States. To visualize other land cover classes, simply change the `y` parameter in the `bar_chart()` function to the desired class code.

[Insert Figure 12.5 here]

We can also visualize the same data as a pie chart:

```

1 fig = geemap.pie_chart(df_stats,
2                         names="STUSPS",
3                         values="Class_82",
4                         title="Cropland Area by State",
5                         legend_title="State",
6                         height=600,
7                         max_rows=30
8 )
9 fig.write_image("gee_cropland_pie_chart.svg")
10 fig

```

This pie chart (Figure 12.6) shows:

- The proportion of US cropland in each state
- State abbreviations as labels
- A legend with state names
- A height of 600 pixels to accommodate the detailed breakdown
- Maximum of 30 rows in the legend to ensure all states are visible

[Insert Figure 12.6 here]

We can clearly see that Iowa, Kansas, and North Dakota have the largest cropland areas, while states like New York and Pennsylvania have significantly less. The pie chart provides a clear visual representation of the relative proportions of cropland in each state, making it easy to identify which states dominate in agricultural land use.

Together, these visualizations provide complementary views of agricultural land distribution: the bar chart emphasizes the absolute area of cropland in each state, while the pie chart highlights the relative proportion of cropland among states.

1.4.3 Estimating Building Heights with Planetary Computer

In this section, we demonstrate how to estimate building heights using LiDAR data from Microsoft's Planetary Computer and building footprint data from Overture Maps. This integration of diverse geospatial datasets enables detailed 3D urban analysis.

1.4.3.1 Import Libraries

First, we import the necessary modules:

```

1 import leafmap
2 from leafmap.download import (
3     pc_stac_search,
4     pc_stac_download,
5     read_pc_item_asset,
6     sign_pc_item,
7     extract_building_stats,
8     get_overture_data,
9 )

```


These imports include specialized functions from leafmap's download module:

- `pc_stac_search`: Searches for data in the Planetary Computer catalog using the STAC API
- `pc_stac_download`: Downloads data from Planetary Computer
- `read_pc_item_asset`: Loads a specific asset from a Planetary Computer item
- `sign_pc_item`: Generates authenticated URLs for Planetary Computer data access
- `get_overture_data`: Retrieves building footprint data from Overture Maps
- `extract_building_stats`: Extracts statistics from building footprint data

1.4.3.2 Defining the Study Area

We'll define a bounding box for our study area near Washington DC:

```
1 m = leafmap.Map(center=[38.8343, -77.1632], zoom=13)
2 m
```

This creates an interactive map centered on Arlington, Virginia, near Washington DC. This location contains a mix of building types and heights, making it a good example for height analysis.

We can either draw a region of interest on the map or use a predefined bounding box:

```
1 bbox = m.user_roi_bounds()
2 if bbox is None:
3     bbox = [-77.2066, 38.8274, -77.1847, 38.8364]
```

This code:

1. Attempts to get a user-drawn region of interest using `user_roi_bounds()`
2. If no region was drawn, falls back to a predefined bounding box
3. The bounding box coordinates represent [west, south, east, north] boundaries in decimal degrees

This approach gives users flexibility in selecting their study area while ensuring the code runs smoothly even without user input.

1.4.3.3 Downloading LiDAR Height Above Ground (HAG) Data

Now we search for and download LiDAR-derived Height Above Ground (HAG) data:

```
1 items = pc_stac_search(
2     collection="3dep-lidar-hag",
3     bbox=bbox,
4     time_range="2018-01-01/2018-12-31",
5     limit=1
6 )
```

This function:

1. Queries the Planetary Computer catalog for data from the 3dep-lidar-hag collection
2. Filters for data that intersects our bounding box
3. Limits the search to data collected in 2018
4. Returns at most one item to keep the example focused

The 3DEP (3D Elevation Program) LiDAR HAG dataset is derived from high-resolution LiDAR point clouds collected by the USGS. The HAG data specifically represents the height of objects above the ground surface, making it ideal for extracting building heights.

Let's examine the metadata of the first item found:

```
1 items[0]
```

This displays detailed metadata about the dataset, including:

- A unique identifier
- Temporal and spatial information about when and where the data was collected
- Available data assets (e.g., the HAG raster, thumbnails, metadata files)
- Data quality indicators and processing information

Understanding this metadata helps us assess the dataset's suitability for our analysis.

Now we can visualize this data on the map:

```
1 m = leafmap.Map()
2 m.add_basemap("Esri.WorldImagery")
3 m.add_stac_layer(
4     collection="3dep-lidar-hag",
5     item=items[0].id,
6     assets=["data"],
7     name="Height Above Ground (HAG)",
8 )
9 m
```

The map displays the HAG data as a raster overlay on satellite imagery. Brighter areas typically represent taller objects (buildings, trees), while darker areas represent ground level. This visual inspection helps us assess the data quality and coverage.

Alternatively, you can download the data to your local machine for offline analysis:

```
1 pc_stac_download(
2     items[0],
3     output_dir="data",
4     assets=["data"],
5 )
```

We can generate a signed URL for accessing the data programmatically:

```

1 href = sign_pc_item(
2     items[0],
3     asset="data",
4 )
5 href

```

This creates a temporary authenticated URL that provides access to the data even from scripts running outside the notebook.

Alternatively, we can load the HAG data directly into memory:

```

1 hag = read_pc_item_asset(items[0], asset="data")

```

This reads the HAG data into an xarray DataArray, which can be used for further analysis in Python.

1.4.3.4 Downloading Building Footprint Data

Next, we download building footprint data from Overture Maps for the same area:

```

1 gdf = get_overture_data(
2     overtуре_type="building",
3     bbox=bbox,
4     columns=["height", "geometry"]
5 )

```

This function:

1. Connects to the Overture Maps database
2. Requests building footprints within our defined bounding box
3. Retrieves only the specified columns (**height** and **geometry**) to minimize data transfer
4. Returns the data as a GeoDataFrame

Overture Maps is an open data collaboration that provides global building footprints with various attributes. The **height** attribute is populated for some buildings based on existing data sources, but many buildings lack height information, which is why we'll use the LiDAR data to fill these gaps.

Let's examine the building data:

```

1 gdf.head()

```

This displays the first few rows of the GeoDataFrame, showing:

- The height attribute (when available, often null)
- The geometry column containing polygon shapes that represent building footprints

We save the building data for later use:

```

1 gdf.to_file("buildings.geojson")

```

This saves the building footprints as a GeoJSON file, which can be shared or used in other GIS applications.

1.4.3.5 Analyzing Building Statistics

Now we extract basic statistics about the buildings in our study area:

```
1 stats = extract_building_stats(gdf)
2 print(stats)
```

This function analyzes the building footprint data and returns key statistics like the total number of buildings, and the number of buildings with height information.

Let's add the building footprints to our map:

```
1 m.add_gdf(gdf, layer_name="Buildings")
2 m
```

This adds the building footprint polygons to our map as an overlay. Use the layer control panel to toggle the visibility of the building layer, HAG layer, and basemaps.

1.4.3.6 Calculate Building Heights from LiDAR Data

Now we use zonal statistics to calculate height statistics for each building from the LiDAR data:

```
1 stats = leafmap.zonal_stats(gdf, href, stats=["min", "max", "mean",
2     , "median"], gdf_out=True)
```

This function:

1. Uses the building footprints (polygons) from our GeoDataFrame
2. For each building, extracts all HAG pixel values that fall within its footprint
3. Calculates statistical measures for those values:
 - Min: Lowest height value (often near zero due to edge effects or ground points)
 - Max: Highest height value (typically representing the roof peak or highest structure)
 - Mean: Average height across the building footprint
 - Median: Middle value, often more representative for buildings with complex roofs
4. Returns the original GeoDataFrame with these statistics added as new columns

This operation effectively extracts 3D information (height) from the 2D building footprints by incorporating the LiDAR-derived height data.

```
1 stats
```

This displays the GeoDataFrame with the newly added height statistics columns. We can now see the height characteristics of each building in our dataset.

Let's visualize the distribution of building heights:

```
1 import matplotlib.pyplot as plt
2
3 ax = stats['median'].hist()
4 plt.xlabel('Height (m)')
5 plt.ylabel('Frequency')
6 plt.title('Histogram of Building Height')
7
8 plt.savefig("building_height_hist.svg", format="svg", bbox_inches=
9             'tight')
10 plt.show()
```

This creates a histogram (Figure 12.7) showing the distribution of median building heights in our study area. The distribution might reveal patterns like:

- Most buildings clustering around certain heights (e.g., single-family homes around 5-10 meters)
- Taller buildings forming a separate cluster or long tail
- Bimodal or multimodal distributions suggesting different building types

[Insert Figure 12.7 here]

Now we update our GeoDataFrame with the median height values:

```
1 gdf['height'] = stats['median']
```

This adds or updates the 'height' column in our original GeoDataFrame with the median height values calculated from the LiDAR data. We use the median rather than the mean because it's less influenced by outliers or noise in the LiDAR data.

```
1 building_height = "building_height.geojson"
2 gdf[['height', 'geometry']].to_file(building_height)
```

This code saves the data as a GeoJSON file. The resulting file contains building footprints with their estimated heights, ready for 3D visualization or further analysis.

1.4.4 Visualizing Building Height Data in 3D

In this final section, we create interactive 3D visualizations of building data. 3D visualization helps urban planners, architects, and analysts understand the vertical dimension of the built environment.

```
1 import leafmap.maplibregl as leafmap
```

This imports the maplibregl submodule of leafmap, which provides 3D mapping capabilities using the MapLibre GL JS library. Unlike the standard leafmap

module which is based on ipyleaflet, the maplibregl submodule enables advanced 3D visualizations including building extrusions.

1.4.4.1 2D Visualization of Building Heights

First, let's create a 2D map with buildings colored by height:

```
1 m = leafmap.Map()
2 m.add_data(building_height, column="height")
3 m
```

This creates a 2D thematic map where:

1. Building footprints are displayed as filled polygons
2. The color of each building corresponds to its height
3. A color scale (legend) shows the mapping between colors and height values

This 2D visualization provides a quick overview of height patterns across the study area. Taller buildings typically appear in dark blue colors, while shorter buildings appear in white colors.

1.4.4.2 3D Extrusion of Buildings

Now we create a 3D visualization by extruding buildings based on their height:

```
1 m = leafmap.Map(pitch=60, bearing=30, projection="globe")
2 m.add_data(building_height, column="height", cmap="Blues", extrude
3             =True)
3 m
```

This creates a 3D visualization (Figure 12.8) where:

1. Buildings are extruded (given height) based on their median HAG value
2. The map view is tilted (pitch=60 degrees) and rotated (bearing=30 degrees) to provide a 3D perspective
3. Buildings are colored using the "Blues" colormap, with darker blues for taller buildings
4. The extrude=True parameter is the key setting that creates the 3D effect

[Insert Figure 12.8 here]

This 3D visualization makes it much easier to identify tall buildings and understand the vertical structure of the urban environment. Users can navigate this map interactively, changing the viewpoint to explore different perspectives on the urban morphology.

For comparison, let's also look at a different area with pre-existing building height data:

```
1 m = leafmap.Map(
2     center=[-74.0095, 40.7046], zoom=16, pitch=60, bearing=-17,
3     style="positron", projection="globe"
4 )
4 m.add_basemap("Esri.WorldImagery", visible=False)
```

```

5 m.add_overture_3d_buildings(template="simple")
6 m.add_layer_control()
7 m

```

This code creates a 3D visualization of buildings in Lower Manhattan, New York City (Figure 12.9):

1. Centers the map on the Financial District area
2. Sets an appropriate zoom level, pitch, and bearing for 3D viewing
3. Uses the `positron` basemap style for a clean background
4. Adds satellite imagery as an optional basemap (default hidden)
5. Uses the `add_overture_3d_buildings` function to add 3D buildings from Overture Maps
6. Adds a layer control to toggle between different layers

[Insert Figure 12.9 here]

The `template="simple"` parameter uses a default popup template for the buildings, which can be customized to show additional information like building names, heights, and other attributes. The resulting 3D view provides an intuitive representation of the iconic Manhattan skyline.

1.4.4.3 Application Summary

The three applications we explored in this section demonstrate the power of geospatial analysis and visualization techniques in Python. By leveraging libraries like Geemap, Leafmap, and open-access data sources (e.g., Google Earth Engine, Planetary Computer, and Overture Maps), we can perform complex analyses and create compelling visualizations that enhance our understanding of spatial data. These tools enable us to tackle real-world challenges in urban planning, environmental monitoring, and disaster response, making geospatial analysis more accessible and impactful.

1.5 BEST PRACTICES AND OPEN ISSUES

1.5.1 Vector Data Visualization Best Practices

Selecting the appropriate format for vector data is critical for efficient visualization and analysis in geospatial applications. The Cloud-Native Geospatial Formats Guide provides comprehensive recommendations for optimal data handling across various use cases. For small vector datasets, GeoJSON remains the preferred format due to its simplicity, wide support, and human-readable structure. However, as datasets grow in size and complexity, alternatives become necessary. Large vector datasets benefit from formats like GeoJSONL (for streaming processing), GeoPackage (for local storage with spatial indexing), GeoParquet (for distributed analysis), or FlatGeoBuf (for efficient random access). When optimizing for cloud storage and analytical queries, GeoParquet stands out by combining columnar storage with spatial indexing capabilities, en-

abling efficient filtering and aggregation operations across distributed computing environments.

Visualization of vector data introduces additional considerations, particularly when working with web maps that must maintain responsiveness across different zoom levels and extents. PMTiles has emerged as a particularly effective solution for visualizing large vector datasets, offering progressive loading and efficient caching mechanisms. This format enables smooth visualization experiences even when working with millions of features. Beyond format selection, geometry simplification represents another crucial optimization technique. By reducing vertex counts while preserving essential shape characteristics, simplified geometries dramatically reduce file sizes and improve rendering performance. This process should be applied dynamically based on the visualization scale – using more detailed geometries at closer zoom levels and simplified versions for overview maps. For processing extensive vector datasets, combining GeoPandas with distributed computing frameworks like Dask or Apache Sedona enables scalable operations that would otherwise be impossible on a single machine.

1.5.2 Raster Data Visualization Best Practices

Cloud Optimized GeoTIFFs (COGs) have revolutionized how we work with large raster datasets by enabling efficient, partial access to remote data through HTTP range requests. This format organizes data into internal tiles with overview levels, allowing applications to retrieve only the specific portions needed for visualization at the current zoom level. Tools like leafmap's `cog_validate()` help ensure raster datasets conform to the COG specification, while `image_to_cog()` provides a straightforward way to convert conventional GeoTIFFs to this optimized format. When working with extensive collections of raster data, SpatioTemporal Asset Catalogs (STAC) provide a standardized approach for organizing, discovering, and accessing geospatial assets. This metadata specification facilitates efficient searching across multiple dimensions including spatial extent, temporal range, and data characteristics.

For analysis and manipulation of raster data, rasterio offers a Pythonic interface to the GDAL library, while rioarray extends these capabilities by integrating with the powerful labeled multidimensional array operations of xarray. This combination is particularly valuable when working with time series of raster data or multi-dimensional scientific datasets. When processing exceeds the capacity of a single machine, frameworks like Dask enable parallel operations through chunked arrays, while the zarr format provides efficient storage and access for distributed computing. These technologies make it possible to visualize and analyze terabyte-scale datasets that would otherwise be unmanageable. Effective visualization of raster data also requires thoughtful color mapping and contrast enhancement to highlight relevant patterns and features while maintaining perceptual accuracy – considerations that become increasingly important when visualizing multispectral or hyperspectral imagery (?).

1.5.3 Challenges and Limitations

Despite significant advances in geospatial visualization technologies, several persistent challenges limit their effectiveness in critical applications. Access to high-resolution satellite imagery remains constrained during disaster response scenarios when such data is most urgently needed. Complex licensing agreements, costs, and technical barriers often prevent timely distribution of imagery to emergency responders and affected communities when every minute counts. While Google Earth Engine provides extraordinary capabilities for large-scale geospatial analysis, its proprietary nature raises concerns about long-term accessibility and vendor lock-in. The recent discontinuation of Microsoft's Planetary Computer JupyterHub highlights the vulnerability of relying on corporate platforms for essential research infrastructure. Open-source alternatives exist but typically require substantial technical expertise to deploy and maintain.

Computational resources present another significant limitation, particularly for advanced visualization and analysis techniques. GPU acceleration has become essential for deep learning applications and complex 3D visualizations, yet access to these resources remains limited in free computing environments like Google Colab. This creates an accessibility gap that disproportionately affects researchers and practitioners with limited institutional resources. The geospatial AI ecosystem, while growing rapidly, still lacks user-friendly packages that make advanced techniques accessible to domain experts without extensive programming experience. This gap between cutting-edge research and practical applications slows the adoption of innovative approaches in real-world scenarios.

Support for specialized data types represents a final frontier in geospatial visualization. Three-dimensional point clouds from LiDAR and photogrammetry contain rich information but remain challenging to visualize and analyze efficiently, especially at large scales. Time series visualization presents similar challenges when attempting to represent both spatial and temporal patterns simultaneously. As these data types become increasingly common in environmental monitoring, urban planning, and climate science, developing more effective visualization techniques will be essential for extracting meaningful insights from complex spatiotemporal datasets.

1.6 FUTURE IMPLICATIONS

The future of geospatial technology is inextricably linked to advancements in artificial intelligence, promising a paradigm shift in how we analyze, visualize, and interact with spatial data. Large Language Models (LLMs) are poised to revolutionize geospatial workflows, enabling natural language queries for complex analysis, automated report generation, and intelligent data exploration. Complementing this, the development of geospatial foundation models—AI models specifically trained on vast geospatial datasets—will provide a powerful base for

diverse applications, from automated feature extraction to predictive modeling. Enhancing accessibility is also paramount, with the rise of low-code and no-code platforms empowering users of all skill levels to harness the power of geospatial analysis. A thriving Jupyter ecosystem, with specialized extensions and seamless geospatial data integration, such as JupyterGIS, will further streamline development and collaboration. Lastly, the emergence of community-driven geospatial data catalogs will be critical for ensuring open access to the wealth of geospatial information, fostering innovation and collaborative problem-solving. These converging trends herald a future where geospatial technology is more intelligent, accessible, and collaborative, driving impactful solutions across diverse domains.

BIBLIOGRAPHY

- Bossche, J. V., Jordahl, K., Fleischmann, M., Richards, M., McBride, J., Wasserman, J., Badaracco, A. G., Snow, A. D., Ward, B., Tratner, J., Gerard, J., Perry, M., cjof, Hjelle, G. A., Taves, M., ter Hoeven, E., Cochran, M., Bell, R., rraymondgh, Bartos, M., Roggemans, P., Culbertson, L., Caria, G., Tan, N. Y., Eubank, N., sangarshanan, Flavin, J., Rey, S., and Gardiner, J. (2024). `geopandas/geopandas: v1.0.1`.
- Cui, S., Gao, Y., Huang, Y., Shen, L., Zhao, Q., Pan, Y., and Zhuang, S. (2023). Advances and applications of machine learning and deep learning in environmental ecology and health. *Environmental pollution (Barking, Essex: 1987)*, 335(122358):122358.
- Gillies, S. (2019). rasterio Documentation. *MapBox*, July, 23.
- Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., and Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote sensing of environment*, 202:18–27.
- Li, Y., Zhang, H., Xue, X., Jiang, Y., and Shen, Q. (2018). Deep learning for remote sensing image classification: A survey. *Wiley interdisciplinary reviews. Data mining and knowledge discovery*, 8(6):e1264.
- Liu, B. and Wu, Q. (2024). HyperCoast: A python package for visualizing and analyzing hyperspectral data in coastal environments. *The Journal of open source software*, 9(100):7025.
- Oscio, L. P., Wu, Q., de Lemos, E. L., Gonçalves, W. N., and others (2023). The segment anything model (sam) for remote sensing applications: From zero to one shot. *International Journal of Applied Earth Observation and Geoinformation*, (124):103540.
- Tufte, E. R. (2001). *The visual display of quantitative information*. Graphics Press.
- Wu, Q. (2020). `geemap`: A Python package for interactive mapping with Google Earth Engine. *The Journal of Open Source Software*, 5(51):2272.
- Wu, Q. (2021). `Leafmap`: A Python package for interactive mapping and geospatial analysis with minimal coding in a Jupyter environment. *The Journal of Open Source Software*, 6(63):3414.
- Wu, Q. and Oscio, L. P. (2023). `samgeo`: A Python package for segmenting geospatial data with the Segment Anything Model (SAM). *The Journal of Open Source Software*.